Spatialized Prisoner's Dilemma

Assignment 2

Braeden Kloke

SYSC 5104: Methodologies for Discrete Event Modeling and Simulation

Department of Systems and Computer Engineering

Carleton University

Dr. Gabriel Wainer

March 31, 2026

1. Conceptual model

The Prisoner's Dilemma is a widely studied model in game theory. It describes two players each of whom can "cooperate" for mutual benefit or "defect" for personal gain. When implemented as cellular automata, it is referred to as the *Spatialized Prisoner's Dilemma*. This model can be used to study how cooperation emerges in a society of selfish actors [1].

Prisoners can have a variety of *reactive strategies* where their next move depends on their opponent's previous move [1]. Table 1 describes these strategies, where:

- *i* represents the prisoner's initial move
- *c* represents how the prisoner reacts to an opponent cooperating
- *d* represents how the prisoner reacts to an opponent defecting

This model focuses on the always defect (AllD) and always cooperate (AllC) strategies to see which of these strategies emerge dominant within a tournament.

Table 1: Prisoner reactive strategies.

i	*c*	*d*	*reactive strategy*
0	0	0	Always defect (AllD)
0	0	1	Suspicious doormat
0	1	0	Suspicious Tit for Tat
0	1	1	Suspicious cooperation
1	0	0	Deceptive defector
1	0	1	Gullible doormat
1	1	0	Tit for Tat (TFT)
1	1	1	Always cooperate (AllC)

2. Formal specification

Table 2 describes the Cell-DEVS definition for a Player. For clarity, the definition focuses on the cell's state variables (θ) and the local computing function (τ). The cell's state variable *cooperate* is True when the player cooperates and False when the player defects. The variable *total_payoff* represents the sum of all the payoffs the player received throughout all the rounds of a tournament. The local computation involves the cell playing a round of the Prisoner's Dilemma with each of its neighbors. This definition implicitly implements the AllD and AllC strategies described in Table 1.

Table 2: Cell-DEVS definition for a Player.

$\theta =$	$\{cooperate \in \{True, False\}, total\_payoff \in \mathbb{Z}^+\}$
$\tau =$	$for\ n\ in\ neighborhood:$ $\quad if\ self.cooperate = True\ and\ n.cooperate = True:$ $\quad\quad total\_payoff = total\_payoff + REWARD;$ $\quad else\ if\ self.cooperate = True\ and\ n.cooperate = False:$ $\quad\quad total\_payoff = total\_payoff + SUCKERS\_PAYOFF;$ $\quad else\ if\ self.cooperate = False\ and\ n.cooperate = True:$ $\quad\quad total\_payoff = total\_payoff + TEMPTATION\_PAYOFF;$ $\quad else:$ $\quad\quad total\_payoff = total\_payoff + PUNISHMENT;$

3. Testing and validation

The Cell-DEVS model in Table 2 was implemented in Cadmium. Their payoffs were implemented using the classic payoff matrix shown in Table 3. Three tests were performed to validate the model. Each test validates a unique outcome in the payoff matrix: either both players cooperate, one cooperates and the other defects, or both players defect. The logs from these tests are shown respectively in Figure 1, Figure 2, and Figure 3. Logs displays the prisoner's data as a tuple represented as $\langle cooperate, total\_payoff \rangle$.

Table 3: Classic payoff matrix.

		Player A	
		Cooperate	Defect
Player B	Cooperate	3, 3	0, 5
	Defect	5, 0	1, 1

```
time;model_id;model_name;port_name;data
0;1;(0,1);;<1,0>
0;2;(0,0);;<1,0>
0;1;(0,1);outputNeighborhood;<1,0>
0;1;(0,1);;<1,3>
0;2;(0,0);outputNeighborhood;<1,0>
0;2;(0,0);;<1,3>
0;1;(0,1);;<1,3>
0;2;(0,0);;<1,3>
```

Figure 1: Cooperator vs. cooperator test log. Both prisoners end up with the reward payoff after one round.

```
time;model_id;model_name;port_name;data
0;1;(0,1);;<0,0>
0;2;(0,0);;<1,0>
0;1;(0,1);outputNeighborhood;<0,0>
0;1;(0,1);;<0,5>
0;2;(0,0);outputNeighborhood;<1,0>
0;2;(0,0);;<1,0>
0;1;(0,1);;<0,5>
0;2;(0,0);;<1,0>
```

Figure 2: Cooperator vs defector test log. Defector ends with the temptation payoff while the cooperator ends with the sucker's payoff after one round.

```
time;model_id;model_name;port_name;data
0;1;(0,1);;<0,0>
0;2;(0,0);;<0,0>
0;1;(0,1);outputNeighborhood;<0,0>
0;1;(0,1);;<0,1>
0;2;(0,0);outputNeighborhood;<0,0>
0;2;(0,0);;<0,1>
0;1;(0,1);;<0,1>
0;2;(0,0);;<0,1>
```

Figure 3: Defector vs. defector test log. Both prisoners receive the punishment payoff after one round.

4. Experimental frame

Table 4 describes the experimental frame.

Table 4: Experiment frame.

Number of rounds per experiment	200, as per Axelrod's Tournament [2]
Payoff matrix	Classic (see Table 3)
Neighborhood	Von Neumann
Cell space	4 x 4
Wrapped	False

5. Experimental results

Experiments were run iteratively to see which strategy emerges dominant, either AllD or AllC. As described in the experimental frame, players play 200 rounds of the Prisoner's Dilemma with each their neighbors. After each experiment, the player adopts the strategy of the neighbor with the highest total payoff. The cell space is randomized to contain an even split between players whose strategies are AllD or AllC.

Results from the experiments are shown in Figure 4, Figure 5, and Figure 6. The left matrix shows the layout of players and their strategies, black squares represent players with the strategy AllD and white squares represent players with the strategy AllC. The right matrix represents the total payoff accumulated throughout the experiment where darker squares represent a higher total payoff.

The results from the experiments show how the AllD strategy eventually dominates the AllC strategy.

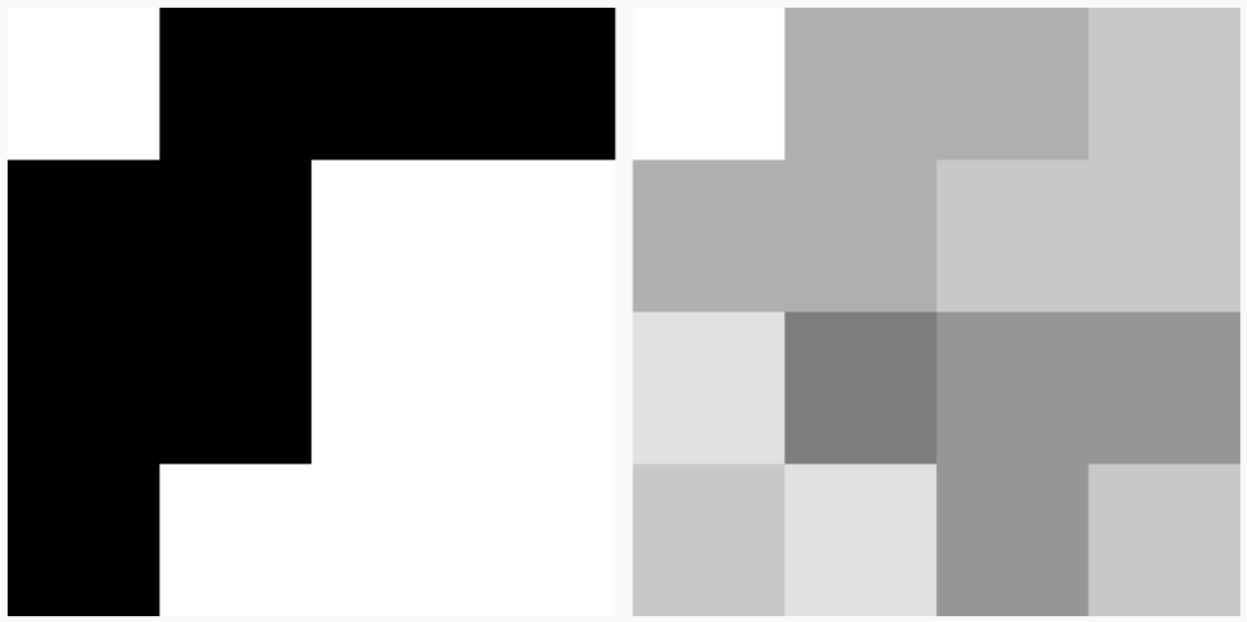

Figure 4: Experiment 1 results. (left) Player strategies: white squares are AllC and black squares are AllD. (right) Total payoff after 200 rounds, darker squares represent higher total payoffs.

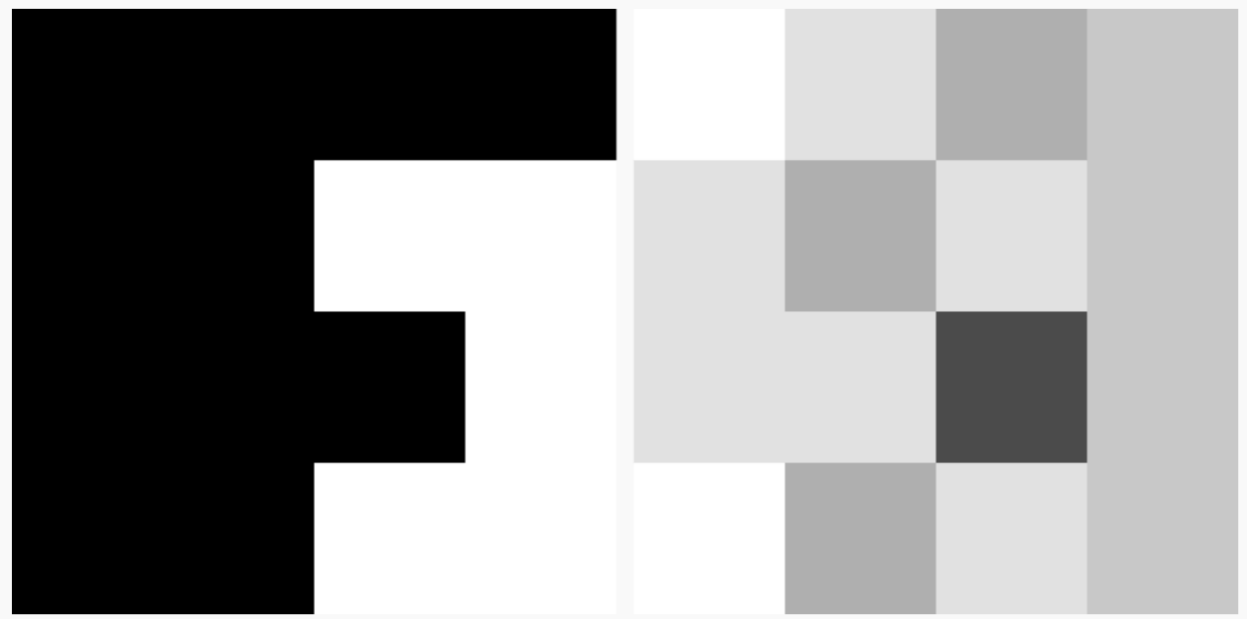

Figure 5: Experiment 2 results.

Figure 6: Experiment 3 results.

References

[1] P. Grim, G. Mar and P. St. Denis, The Philosophical Computer : Exploratory Essays in Philosophical Computer Modeling, MIT Press, 1998.

[2] R. Axelrod, "Effective Choice in the Prisoner's Dilemma," *The Journal of Conflict Resolution,* vol. 24, no. 1, pp. 3-25, 1980.